

Proyecto: Cafetería - Documentación Técnica

Programación de Servicios y Procesos

Adrián Condines Celada

Aula Estudio

2º Ciclo Superior - Desarrollo de Aplicaciones Multiplataforma

Curso 2025 - 2026

Índice

1. Introducción y Objetivos	2
2. Stack Tecnológico	2
2.1. Java	2
2.2. JavaFX	2
2.3. CSS	3
2.4. Visual Studio Code	3
3. Requisitos Funcionales	4
4. Requisitos No Funcionales	4
5. Casos de Uso	5
6. Historias de Usuario	5
7. Estructura del Proyecto	6
8. Clases Principales	7
8.1. Clase Cliente	7
8.1.1. Descripción	7
8.1.2. Atributos	7
8.1.3. Métodos	7
8.2. Clase Camarero	8
8.2.1. Descripción	8
8.2.2. Atributos	8
8.2.3. Métodos	8
9. Interfaz	9

1. Introducción y Objetivos

El objetivo de esta práctica es desarrollar una aplicación en Java que simule el funcionamiento de una cafetería en la que los clientes llegarán periódicamente a la cafetería a realizar su pedido y serán atendidos por los camareros disponibles.

La aplicación debe gestionar la llegada de clientes, la atención por parte de los camareros y la salida de los clientes una vez que han sido atendidos.

2. Stack Tecnológico

La aplicación se desarrollará utilizando las siguientes tecnologías y herramientas:

2.1. Java



El lenguaje de programación utilizado para desarrollar la lógica de la aplicación es Java, debido a su robustez y capacidad para manejar múltiples hilos de ejecución.

2.2. JavaFX



JavaFX se ha utilizado como tecnología para desarrollar la interfaz gráfica de la aplicación, proporcionando una manera visual de ver el resultado del programa.

2.3. CSS



CSS se ha utilizado para dar estilo a la interfaz gráfica desarrollada con JavaFX, mejorando la experiencia del usuario.

2.4. Visual Studio Code



Como IDE se ha utilizado Visual Studio Code, que gracias a sus extensiones permite un desarrollo eficiente en Java y JavaFX.

3. Requisitos Funcionales

La aplicación debe cumplir con los siguientes requisitos funcionales:

RF-1: Cada cliente debe ejecutarse en un hilo independiente que simule su llegada a la cafetería.

RF-2: Cada cliente debe intentar pedir un café y esperar un tiempo determinado (`tiempoEspera`).

RF-3: Si el café no está preparado antes de que se cumpla su `tiempoEspera`, el cliente se irá y se reflejará con un mensaje.

RF-4: Si el cliente recibe su café a tiempo, se reflejará con un mensaje que incluya que el cliente ha sido servido y cuanto tiempo ha tomado en ser atendido.

RF-5: Cada camarero debe ejecutarse en un hilo independiente que simule la atención a los clientes.

RF-6: Los camareros deben preparar los pedidos de los clientes en base al orden de llegada.

RF-7: Si un cliente se va antes de que su café esté listo, el camarero debe cancelar la preparación del café y pasar al siguiente cliente en la cola.

RF-8: El programa debe de seguir ejecutándose hasta que todos los clientes hayan sido atendidos o se hayan ido.

4. Requisitos No Funcionales

La aplicación debe cumplir con los siguientes requisitos no funcionales:

RNF-1: El programa debe de manejar múltiples hilos de ejecución para simular la concurrencia entre clientes y camareros.

RNF-2: Se debe garantizar que cada cliente sea atendido una sola vez.

RNF-3: Los mensajes en la consola deben ser claros y proporcionar información relevante sobre el estado de la cafetería.

RNF-4: El código debe estar bien estructurado y documentado para facilitar su comprensión y mantenimiento.

Casos de Uso	Descripción
Iniciar aplicación	El usuario inicia la aplicación para simular el funcionamiento de una cafetería.

5. Casos de Uso

A continuación se describen los casos de uso principales de la aplicación:

6. Historias de Usuario

A continuación se describen las historias de usuario principales de la aplicación:

ID	Como un...	Quiero...	Para...
HU-1	Usuario	Iniciar la aplicación	Simular el funcionamiento de una cafetería con clientes y camareros.

7. Estructura del Proyecto

La estructura del proyecto está organizada de la siguiente manera:

```
Cafetería/
|-- DOCUMENTACIÓN/
|   |-- images/
|       |-- css_icon.png
|       |-- java_icon.png
|       |-- javafx_icon.png
|       |-- vscode_icon.png
|   |-- DOCUMENTACIÓN_CAFETERÍA - ADRIANO.tex
|   |-- DOCUMENTACIÓN_CAFETERÍA - ADRIANO.pdf
|-- src/main
|   |-- java/
|       |-- com/akadoble/cafeteria/
|           |-- Cliente.java
|           |-- Camarero.java
|           |-- Main.java
|           |-- MainController.java
|   |-- resources/
|       |-- MainView.fxml
|       |-- style.css
```

8. Clases Principales

8.1. Clase Cliente

8.1.1. Descripción

La clase `Cliente` representa a un cliente que llega a la cafetería para pedir un café.

8.1.2. Atributos

Los atributos de la clase `Cliente` son los siguientes:

- `String nombre`: Nombre del cliente.
- `String tiempoEspera`: Tiempo máximo que el cliente está dispuesto a esperar por su café.
- `boolean atendido`: Indica si el cliente ha sido atendido o no.
- `long inicioEspera`: Marca de tiempo que indica cuándo comenzó a esperar el cliente.
- `Queue<Cliente> colaClientes`: Cola compartida de clientes esperando ser atendidos.
- `MainController controlador`: Referencia al controlador.

8.1.3. Métodos

Los métodos de la clase `Cliente` son los siguientes:

- `esperarLlegadaAleatoria()`: Simula la llegada del cliente a la cafetería después de un tiempo aleatorio.
- `registrarLlegada()`: Registra la llegada del cliente y lo comunica al controlador.
- `irsePorImpaciencia()`: Maneja el caso en que el cliente se cansa de esperar y se va.
- `dormirHilo()`: Duerme el hilo para evitar bucles de espera.

8.2. Clase Camarero

8.2.1. Descripción

La clase `Camarero` representa a un camarero que atiende a los clientes en la cafetería.

8.2.2. Atributos

- `String nombre`: Nombre del camarero.
- `Queue<Cliente> colaClientes`: Cola compartida de clientes esperando ser atendidos.
- `MainController controlador`: Referencia al controlador.

8.2.3. Métodos

Los métodos de la clase `Camarero` son los siguientes:

- `obtenerSiguienteCliente()`: Obtiene el siguiente cliente o espera si no hay clientes.
- `clienteHaEsperadoDemasiado(Cliente cliente)`: Verifica si el cliente ha esperado más de su tiempo de espera.
- `manejarClienteImpaciente(Cliente cliente)`: Maneja el caso en que un cliente se va antes de ser atendido.
- `prepararCafe(Cliente cliente)`: Simula la preparación del café para el cliente.

9. Interfaz